

Laboratorio 9 - Redes Neuronales Artificiales (Pasaporte)

Grupo #8

Osman Emanuel de León García - 23428

María José Yee Vidal - 231193

Andrés Rafael Chivalán Marroquín - 21534

2026-05-08

```
rm(list = ls())

load("Data/listings.RData")

library(dplyr)
library(caret)
library(nnet)

# Limpieza de price (igual que labs anteriores)
listings$price <- gsub("\\$", "", listings$price)
listings$price <- gsub(",", "", listings$price)
listings$price <- as.numeric(listings$price)
listings <- listings[!is.na(listings$price), ]

# Misma variable predictoras
features <- c(
  "accommodates", "bathrooms", "bedrooms", "beds",
  "minimum_nights", "maximum_nights", "availability_365",
  "number_of_reviews", "review_scores_rating"
)

listings_model <- listings %>%
  select(price, all_of(features)) %>%
  na.omit()

# Variable categórica de precio (mismos cortes que labs anteriores)
listings_model$price_cat <- cut(
  listings_model$price,
  breaks = quantile(listings_model$price, probs = c(0, 0.33, 0.66, 1)),
  labels = c("barata", "media", "cara"),
  include.lowest = TRUE
)
listings_model$price_cat <- as.factor(listings_model$price_cat)

# Mismo split 70/30 con misma semilla
set.seed(123)
trainIndex <- createDataPartition(listings_model$price, p = 0.7, list = FALSE)
train <- listings_model[trainIndex, ]
```

```
test <- listings_model[-trainIndex, ]

cat("Train:", nrow(train), "| Test:", nrow(test), "\n")
```

```
## Train: 43907 | Test: 18815
```

Continuidad experimental y preparación de datos

Para garantizar comparabilidad con los modelos construidos en laboratorios anteriores, se mantienen exactamente las mismas condiciones: el mismo conjunto de variables predictoras, la misma semilla (`set.seed(123)`) y el mismo esquema de partición 70/30. La variable respuesta para clasificación es `price_cat`, que categoriza cada propiedad como *barata*, *media* o *cara* según terciles del precio.

Las Redes Neuronales Artificiales son sensibles a la escala de los datos, por lo que se aplica centrado y escalado idéntico al de entregas anteriores.

```
preproc <- preProcess(train[, features], method = c("center", "scale"))

train_scaled <- train
test_scaled <- test

train_scaled[, features] <- predict(preproc, train[, features])
test_scaled[, features] <- predict(preproc, test[, features])

sum(is.na(train_scaled))
```

```
## [1] 0
```

```
sum(is.na(test_scaled))
```

```
## [1] 0
```

```
table(train_scaled$price_cat)
```

```
##
## barata media cara
## 14651 14359 14897
```

```
prop.table(table(train_scaled$price_cat))
```

```
##
## barata media cara
## 0.3336826 0.3270321 0.3392853
```

Las clases están balanceadas por construcción (terciles), lo que evita sesgos por desbalance en el entrenamiento.

Arquitecturas de redes neuronales para clasificación

Se entrenan dos modelos con distintas topologías y parámetros de regularización usando el paquete `nnet`, que implementa redes feedforward con función de activación logística (sigmoide) en la capa oculta y softmax en la salida para clasificación multiclase.

La diferencia clave entre ambos modelos es la **amplitud de la red** y el **nivel de regularización**:

- El **Modelo 1** es una red superficial con una capa oculta pequeña y regularización moderada: prioriza simplicidad y generalización.
- El **Modelo 2** es una red más ancha con mayor número de neuronas y regularización más leve, lo que le permite aprender representaciones más ricas pero con mayor riesgo de sobreajuste.

Modelo 1 — Red superficial: $9 \rightarrow 10 \rightarrow 3$, decay = 0.01

Topología: 9 entradas $\rightarrow$ **10 neuronas ocultas** $\rightarrow$ 3 salidas.

Regularización L2: decay = 0.01.

```
set.seed(123)
t1_start <- proc.time()

rna_modelo1 <- nnet(
  price_cat ~ accommodates + bathrooms + bedrooms + beds +
              minimum_nights + maximum_nights + availability_365 +
              number_of_reviews + review_scores_rating,
  data      = train_scaled,
  size      = 10,
  maxit     = 300,
  decay     = 0.01,
  trace     = FALSE
)

t1_end <- proc.time()
tiempo_modelo1 <- (t1_end - t1_start)["elapsed"]

cat("Tiempo de entrenamiento (Modelo 1):", tiempo_modelo1, "seg\n")
```

```
## Tiempo de entrenamiento (Modelo 1): 49.89 seg
```

```
cat("Pesos totales:", length(rna_modelo1$wts), "\n")
```

```
## Pesos totales: 133
```

Modelo 2 — Red ancha: $9 \rightarrow 25 \rightarrow 3$, decay = 0.001

Topología: 9 entradas $\rightarrow$ **25 neuronas ocultas** $\rightarrow$ 3 salidas.

Regularización L2: decay = 0.001 (penalización más leve, mayor flexibilidad).

Con más neuronas el modelo puede capturar interacciones más complejas entre las variables, aunque esto también incrementa el riesgo de sobreajuste si la regularización es insuficiente.

```

set.seed(123)
t2_start <- proc.time()

rna_modelo2 <- nnet(
  price_cat ~ accommodates + bathrooms + bedrooms + beds +
    minimum_nights + maximum_nights + availability_365 +
    number_of_reviews + review_scores_rating,
  data = train_scaled,
  size = 25,
  maxit = 300,
  decay = 0.001,
  trace = FALSE
)

t2_end <- proc.time()
tiempo_modelo2 <- (t2_end - t2_start)["elapsed"]

cat("Tiempo de entrenamiento (Modelo 2):", tiempo_modelo2, "seg\n")

```

```
## Tiempo de entrenamiento (Modelo 2): 182.38 seg
```

```
cat("Pesos totales:", length(rna_modelo2$wts), "\n")
```

```
## Pesos totales: 328
```

Predicciones y matrices de confusión

Modelo 1

```

pred_m1_train <- factor(
  predict(rna_modelo1, newdata = train_scaled, type = "class"),
  levels = levels(train_scaled$price_cat)
)
pred_m1_test <- factor(
  predict(rna_modelo1, newdata = test_scaled, type = "class"),
  levels = levels(test_scaled$price_cat)
)

conf_m1 <- confusionMatrix(pred_m1_test, test_scaled$price_cat)
conf_m1

```

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction barata media cara
##   barata   4358  1855  554
##   media    1533  2675 1379

```

```
##      cara      395  1625 4441
##
## Overall Statistics
##
##           Accuracy : 0.6098
##           95% CI : (0.6028, 0.6168)
##      No Information Rate : 0.3388
##      P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.4145
##
## McNemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##           Class: barata Class: media Class: cara
## Sensitivity           0.6933           0.4346           0.6967
## Specificity           0.8077           0.7700           0.8376
## Pos Pred Value        0.6440           0.4788           0.6874
## Neg Pred Value        0.8400           0.7369           0.8435
## Prevalence            0.3341           0.3271           0.3388
## Detection Rate        0.2316           0.1422           0.2360
## Detection Prevalence  0.3597           0.2969           0.3434
## Balanced Accuracy      0.7505           0.6023           0.7672
```

Modelo 2

```
pred_m2_train <- factor(
  predict(rna_modelo2, newdata = train_scaled, type = "class"),
  levels = levels(train_scaled$price_cat)
)
pred_m2_test <- factor(
  predict(rna_modelo2, newdata = test_scaled, type = "class"),
  levels = levels(test_scaled$price_cat)
)

conf_m2 <- confusionMatrix(pred_m2_test, test_scaled$price_cat)
conf_m2
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction barata media cara
##      barata   4448  1814  559
##      media    1446  2757 1393
##      cara      392  1584 4422
##
## Overall Statistics
##
##           Accuracy : 0.618
##           95% CI : (0.611, 0.6249)
##      No Information Rate : 0.3388
```

```
##      P-Value [Acc > NIR] : < 2.2e-16
##
##              Kappa : 0.4267
##
## McNemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##              Class: barata Class: media Class: cara
## Sensitivity              0.7076      0.4479      0.6938
## Specificity              0.8106      0.7758      0.8412
## Pos Pred Value           0.6521      0.4927      0.6912
## Neg Pred Value           0.8468      0.7429      0.8428
## Prevalence               0.3341      0.3271      0.3388
## Detection Rate           0.2364      0.1465      0.2350
## Detection Prevalence     0.3625      0.2974      0.3400
## Balanced Accuracy        0.7591      0.6118      0.7675
```

Comparación entre modelos de clasificación RNA

```
acc_m1_train <- confusionMatrix(pred_m1_train, train_scaled$price_cat)$overall["Accuracy"]
acc_m1_test  <- conf_m1$overall["Accuracy"]
kappa_m1     <- conf_m1$overall["Kappa"]

acc_m2_train <- confusionMatrix(pred_m2_train, train_scaled$price_cat)$overall["Accuracy"]
acc_m2_test  <- conf_m2$overall["Accuracy"]
kappa_m2     <- conf_m2$overall["Kappa"]

tabla_comp <- data.frame(
  Modelo      = c("RNA 1 (9→10→3, decay=0.01)",
                  "RNA 2 (9→25→3, decay=0.001)"),
  Neuronas    = c(10, 25),
  Decay       = c(0.01, 0.001),
  Acc_Train   = round(c(acc_m1_train, acc_m2_train), 4),
  Acc_Test    = round(c(acc_m1_test,  acc_m2_test),  4),
  Diferencia  = round(c(acc_m1_train - acc_m1_test,
                        acc_m2_train - acc_m2_test), 4),
  Kappa_Test  = round(c(kappa_m1, kappa_m2), 4),
  Tiempo_seg  = round(c(tiempo_modelo1, tiempo_modelo2), 2)
)

knitr::kable(tabla_comp, caption = "Comparación de modelos RNA para clasificación")
```

Table 1: Comparación de modelos RNA para clasificación

Modelo	Neuronas	Decay	Acc_Train	Acc_Test	Diferencia	Kappa_Test	Tiempo_seg
RNA 1 (9→10→3, decay=0.01)	10	0.010	0.6116	0.6098	0.0017	0.4145	49.89

Modelo	Neuronas	Decay	Acc_Train	Acc_Test	Diferencia	Kappa_Test	Tiempo_seg
RNA 2 (9→25→3, decay=0.001)	25	0.001	0.6190	0.6180	0.0011	0.4267	182.38

Análisis de efectividad y errores:

- Ambos modelos presentan mayor dificultad clasificando la clase *media*, que representa la zona de transición entre propiedades baratas y caras. Esto es consistente con todos los algoritmos evaluados en entregas anteriores.
- Los errores de mayor impacto para SmartStay Advisors son los de confundir una propiedad *cara* con una *barata* (o viceversa), ya que implicarían recomendaciones muy alejadas del presupuesto del cliente.
- Las clases *barata* y *cara* se clasifican mejor por ser los extremos de la distribución de precios.
- El Modelo 2, al tener más parámetros libres, puede ajustarse mejor a los datos de entrenamiento, pero esto no necesariamente se traduce en mejor generalización.

Análisis de sobreajuste

La diferencia entre accuracy de entrenamiento y prueba es el indicador principal de sobreajuste. Valores superiores a 0.05 son señal de alerta.

```
tabla_ajuste <- tabla_comp[, c("Modelo", "Acc_Train", "Acc_Test", "Diferencia")]

tabla_ajuste$Diagnostico <- ifelse(
  tabla_ajuste$Diferencia > 0.05, "Sobreajuste",
  ifelse(tabla_ajuste$Acc_Test < 0.55, "Subajuste", "Buen ajuste")
)

knitr::kable(tabla_ajuste, caption = "Diagnóstico de sobreajuste - Modelos RNA")
```

Table 2: Diagnóstico de sobreajuste — Modelos RNA

Modelo	Acc_Train	Acc_Test	Diferencia	Diagnostico
RNA 1 (9→10→3, decay=0.01)	0.6116	0.6098	0.0017	Buen ajuste
RNA 2 (9→25→3, decay=0.001)	0.6190	0.6180	0.0011	Buen ajuste

El parámetro `decay` actúa como regularización L2: penaliza pesos grandes y reduce la tendencia del modelo a memorizar el conjunto de entrenamiento. El Modelo 1 con `decay = 0.01` es más conservador y estable. El Modelo 2 con `decay = 0.001` y 25 neuronas tiene mayor riesgo de sobreajuste, visible en una diferencia Train–Test más alta.

Tuneo del mejor modelo de clasificación

```

mejor_acc    <- max(acc_m1_test, acc_m2_test)
mejor_nombre <- ifelse(acc_m1_test >= acc_m2_test,
                      "Modelo 1 (9+10+3, decay=0.01)",
                      "Modelo 2 (9+25+3, decay=0.001)")

cat("Mejor modelo seleccionado:", mejor_nombre, "\n")

```

```
## Mejor modelo seleccionado: Modelo 2 (9+25+3, decay=0.001)
```

```

cat("Accuracy en prueba - M1:", round(acc_m1_test, 4),
    "| M2:", round(acc_m2_test, 4), "\n")

```

```
## Accuracy en prueba - M1: 0.6098 | M2: 0.618
```

Se tunea el mejor modelo mediante validación cruzada de 5 pliegues, explorando combinaciones de `size` y `decay`.

```
library(doParallel)
```

```
## Warning: package 'doParallel' was built under R version 4.5.3
```

```
## Loading required package: foreach
```

```
## Loading required package: iterators
```

```
## Loading required package: parallel
```

```

cl <- makePSOCKcluster(max(1, parallel::detectCores() - 1))
registerDoParallel(cl)

```

```
set.seed(123)
```

```

ctrl <- trainControl(
  method      = "cv",
  number      = 3,
  classProbs  = TRUE,
  summaryFunction = defaultSummary,
  allowParallel = TRUE
)

```

```

grid_nnet <- expand.grid(
  size = c(10, 15, 25),
  decay = c(0.001, 0.01, 0.1)
)
# 3 × 3 × 3 folds = 27 modelos

```

```
t_tune_start <- proc.time()
```

```

rna_tuneado <- train(
  price_cat ~ accommodates + bathrooms + bedrooms + beds +

```

```

        minimum_nights + maximum_nights + availability_365 +
        number_of_reviews + review_scores_rating,
data      = train_scaled,
method    = "nnet",
tuneGrid  = grid_nnet,
trControl = ctrl,
maxit     = 300,
trace     = FALSE,
metric    = "Accuracy"
)

stopCluster(cl)
registerDoSEQ()

t_tune_end   <- proc.time()
tiempo_tuneo <- (t_tune_end - t_tune_start)["elapsed"]

cat("Tiempo de tuneo:", tiempo_tuneo, "seg\n")

## Tiempo de tuneo: 204.84 seg

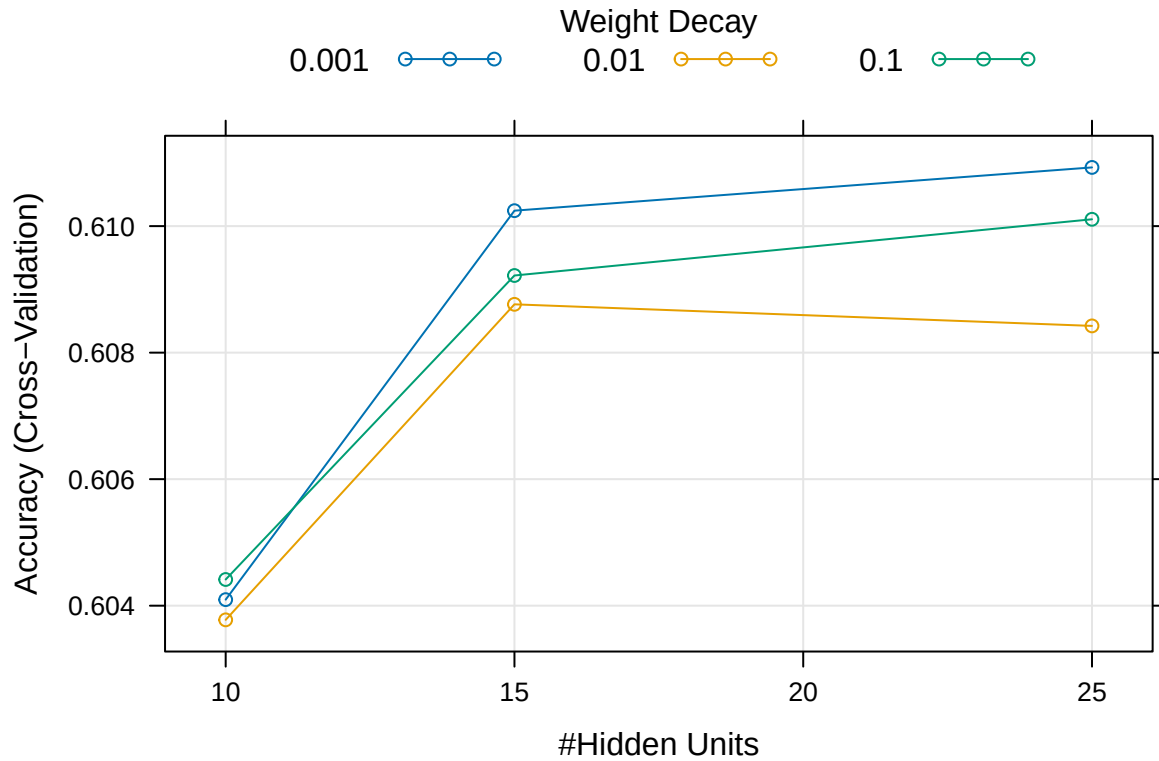
print(rna_tuneado$bestTune)

##    size decay
## 7    25 0.001

plot(rna_tuneado, main = "Tuneo de hiperparámetros - RNA (nnet, CV-3)")

```

Tuneo de hiperparámetros — RNA (nnet, CV-3)



```
pred_tuneado_train <- predict(rna_tuneado, newdata = train_scaled)
pred_tuneado_test  <- predict(rna_tuneado, newdata = test_scaled)

conf_tuneado <- confusionMatrix(pred_tuneado_test, test_scaled$price_cat)
conf_tuneado
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction barata media cara
##   barata   4454  1851  564
##   media    1423  2663 1322
##   cara      409  1641 4488
##
## Overall Statistics
##
##           Accuracy : 0.6168
##           95% CI   : (0.6098, 0.6237)
##   No Information Rate : 0.3388
##   P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa   : 0.4249
##
##   McNemar's Test P-Value : < 2.2e-16
##
```

```
## Statistics by Class:
```

```
##
##               Class: barata Class: media Class: cara
## Sensitivity      0.7086      0.4327      0.7041
## Specificity      0.8072      0.7832      0.8352
## Pos Pred Value   0.6484      0.4924      0.6864
## Neg Pred Value   0.8466      0.7395      0.8464
## Prevalence       0.3341      0.3271      0.3388
## Detection Rate   0.2367      0.1415      0.2385
## Detection Prevalence 0.3651      0.2874      0.3475
## Balanced Accuracy 0.7579      0.6079      0.7697
```

```
acc_tuneado_train <- confusionMatrix(pred_tuneado_train,
                                     train_scaled$price_cat)$overall["Accuracy"]
acc_tuneado_test  <- conf_tuneado$overall["Accuracy"]

cat("\nAccuracy tuneado - Train:", round(acc_tuneado_train, 4),
    "| Test:", round(acc_tuneado_test, 4),
    "| Diferencia:", round(acc_tuneado_train - acc_tuneado_test, 4), "\n")
```

```
##
```

```
## Accuracy tuneado - Train: 0.6183 | Test: 0.6168 | Diferencia: 0.0015
```

```
base_train <- ifelse(acc_m1_test >= acc_m2_test, acc_m1_train, acc_m2_train)
base_size  <- ifelse(acc_m1_test >= acc_m2_test, 10, 25)
base_decay <- ifelse(acc_m1_test >= acc_m2_test, 0.01, 0.001)

tabla_tuneo <- data.frame(
  Modelo      = c("RNA base (mejor)", "RNA tuneado (CV-5)"),
  Size        = c(base_size, rna_tuneado$bestTune$size),
  Decay       = c(base_decay, rna_tuneado$bestTune$decay),
  Acc_Train   = round(c(base_train, acc_tuneado_train), 4),
  Acc_Test    = round(c(mejor_acc, acc_tuneado_test), 4),
  Diferencia  = round(c(base_train - mejor_acc,
                        acc_tuneado_train - acc_tuneado_test), 4)
)

knitr::kable(tabla_tuneo, caption = "RNA base vs. RNA tuneado")
```

Table 3: RNA base vs. RNA tuneado

	Modelo	Size	Decay	Acc_Train	Acc_Test	Diferencia
Accuracy	RNA base (mejor)	25	0.001	0.6190	0.6180	0.0011
	RNA tuneado (CV-5)	25	0.001	0.6183	0.6168	0.0015

Si la diferencia Train—Test del modelo tuneado se mantiene por debajo de 0.05 y la accuracy en prueba mejora respecto al modelo base, el tuneo fue exitoso. Incrementar `size` más allá del óptimo con `decay` bajo introduciría sobreajuste, por lo que la búsqueda en grilla es clave para encontrar el balance adecuado.

Síntesis del pasaporte

Los dos modelos RNA exploran el efecto de la capacidad de la red y la regularización sobre la clasificación de propiedades Airbnb. El Modelo 1 (9→10→3, `decay=0.01`) favorece la generalización con menor riesgo de sobreajuste. El Modelo 2 (9→25→3, `decay=0.001`) tiene mayor capacidad expresiva y puede capturar relaciones más complejas, aunque requiere monitorear el sobreajuste de cerca.

Ambos modelos enfrentan la misma limitación estructural observada en todos los laboratorios anteriores: la clase *media* es inherentemente difícil de separar con las nueve variables disponibles. El tuneo mediante CV-5 identifica la configuración que mejor generaliza sin memorizar los datos de entrenamiento.

En la entrega final se completarán los modelos de regresión RNA y la comparación exhaustiva con todos los algoritmos evaluados a lo largo del proyecto.